

Evaluación Final Transversal

Ejecución práctica

Contexto

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
FPY1101	Fundamentos de programación	4 horas	40%

1.Situación Evaluativa 1:

x	Ejecución práctica
---	--------------------

2.Instrucciones Evaluación Final Transversal

La florería **FlorExpress** requiere un programa en Python para administrar su catálogo de arreglos florales y el stock disponible. Todo el comportamiento del sistema debe organizarse en funciones bien definidas. El programa incluye un menú interactivo, validaciones de entrada y una separación clara entre la lógica de cada función y las decisiones del programa principal.

1. Datos que debe manejar el sistema

El sistema trabaja con **dos diccionarios relacionados**, ambos identificados por el mismo código de arreglo como clave. Estos diccionarios deben existir desde que el programa inicia y permanecer disponibles durante toda la ejecución.

Estos diccionarios deben crearse en el programa principal y pasarse como argumento a todas las funciones que necesiten leerlos o modificarlos (aunque estas no los soliciten en el enunciado). No está permitido acceder a ellos como variables globales dentro de las funciones. También es importante indicar que cada estructura de repetición (while, for) debe utilizarse según su propósito natural (dependiendo de si la condición de parada cambia durante la ejecución o si el volumen de iteraciones se conoce de antemano). Usar una estructura fuera de ese contexto será considerado un error de diseño y no cumplirá con el estándar de la evaluación.

Diccionario arreglos

Contiene la información descriptiva de cada arreglo. La **clave** es el código de arreglo y el **valor** es una lista con los siguientes campos, en este orden:

MALETA DIDÁCTICA DuocUC[®] Restricciones de validación

Campo	Qué representa	Restricciones de validación
"nombre"	Nombre del arreglo floral	No debe contener solo espacios en blanco ni estar vacío
"tipo"	Tipo de arreglo	No debe contener solo espacios en blanco ni estar vacío
"color_principal"	Color principal del arreglo	No debe contener solo espacios en blanco ni estar vacío
"tamaño"	Tamaño del arreglo	Debe ser exactamente 'S', 'M' o 'L'
"incluye_tarjeta"	Indica si el arreglo incluye tarjeta de dedicatoria	El usuario ingresa 's' o 'n'. El sistema lo convierte a True o False
"temporada"	Temporada en que está disponible el arreglo	No debe contener solo espacios en blanco ni estar vacío

Los puntos suspensivos en el siguiente ejemplo indican que pueden existir más registros:

```
arreglos = {  
    'FLO1': ['Ramo Primavera', 'ramo', 'rosado', 'M', True, 'primavera'],  
    'FLO2': ['Caja Elegante', 'caja', 'blanco', 'L', True, 'todo año'],  
    'FLO3': ['Ramo Solar', 'ramo', 'amarillo', 'S', False, 'verano'],  
    'FLO4': ['Centro Mesa', 'centro', 'rojo', 'M', True, 'todo año'],  
    'FLO5': ['Ramo Bosque', 'ramo', 'verde', 'L', False, 'otoño'],  
    'FLO6': ['Caja Noche', 'caja', 'morado', 'M', True, 'invierno'],  
    ...  
}
```

Diccionario bodega

Contiene la información operativa de cada arreglo. La **clave** es el mismo código de arreglo y el **valor** es una lista con los siguientes dos campos:

Campo	Qué representa	Restricciones de validación
-------	----------------	-----------------------------

MALETA DIDÁCTICA DuocUC®

"precio"	Precio de venta del arreglo en pesos	Número entero mayor que cero
"unidades"	Cantidad de unidades disponibles en bodega	Número entero mayor o igual a cero

Los puntos suspensivos en el siguiente ejemplo indican que pueden existir más registros:

```
bodega = {
    'FLO1': [15990, 8],
    'FLO2': [29990, 3],
    'FLO3': [9990, 12],
    'FLO4': [24990, 5],
    'FLO5': [19990, 0],
    'FLO6': [22990, 6],
    ...
}
```

2. Lo que debe hacer el sistema

El sistema se controla desde un menú que aparece en pantalla cada vez que el usuario termina una acción. El usuario elige una opción numérica, el programa ejecuta la tarea correspondiente y vuelve a mostrar el menú. Esto se repite hasta que el usuario elige salir. Si el usuario ingresa un valor que no corresponda a ninguna opción válida, el sistema muestra el mensaje "Debe seleccionar una opción válida" y vuelve a mostrar el menú.

Para la lectura de la opción del menú:

Define una función llamada leer_opcion(). No recibe parámetros. Solicita al usuario que ingrese una opción, valida que el valor ingresado sea un número entero y que esté dentro del rango de opciones válidas del menú, y retorna ese valor entero. Si el usuario ingresa un dato que no es un entero debe manejarlo mediante excepciones.

```
===== MENÚ PRINCIPAL =====
1. Unidades por tipo de arreglo
2. Búsqueda de arreglos por rango de precio
3. Actualizar precio de arreglo
4. Agregar arreglo
5. Eliminar arreglo
6. Salir
=====
```

A continuación, se describe qué debe ocurrir al elegir cada opción:

Opción 1 — Unidades por tipo de arreglo

El sistema solicita al usuario el tipo de arreglo (por ejemplo: ramo, caja o centro). La búsqueda **no distingue entre mayúsculas y minúsculas**, por lo que "ramo" y "RAMO" deben producir el mismo resultado. El sistema recorre el diccionario **arreglos** identificando todos los arreglos que correspondan a ese tipo. Por cada arreglo encontrado, se debe buscar su código en el diccionario **bodega**, extraer la cantidad de unidades disponibles (el segundo elemento de la lista) y acumularla en

un total. Una vez procesados todos los arreglos, se debe mostrar dicho total acumulado en pantalla.

Para implementar esta opción:

Define una función llamada **unidades_tipo(tipo)**. Recibe el tipo de arreglo como parámetro, **no retorna ningún valor** y muestra el resultado directamente por pantalla.

Opción 2 — Búsqueda de arreglos por rango de precio

El sistema solicita al usuario un precio mínimo y un precio máximo. Luego recorre el diccionario **bodega** y construye una lista con todos los arreglos que: (a) tengan un precio dentro del rango ingresado, y (b) tengan unidades disponibles (unidades distinto de cero). Cada elemento de la lista tiene el formato "Nombre--Código". Los resultados se muestran ordenados **alfabéticamente por nombre**. Si no hay arreglos que cumplan las condiciones, el sistema muestra: *"No hay arreglos en ese rango de precios."*

Restricciones de entrada:

El precio mínimo y máximo deben ingresarse como valores enteros. Esta validación ocurre en el **programa principal**, antes de llamar a la función. Como el usuario puede ingresar cualquier tipo de dato, debe utilizarse manejo de excepciones. Si el dato ingresado no es un entero válido, el sistema muestra *"Debe ingresar valores enteros"* y vuelve a solicitar ambos valores.

Para implementar esta opción:

Define una función llamada **busqueda_precio(p_min, p_max)**. Recibe el precio mínimo y máximo como parámetros (*estos valores deben ser mayores o iguales a cero y el p_min menor o igual al p_max*), **no retorna ningún valor** y muestra los resultados directamente por pantalla.

Opción 3 — Actualizar precio de arreglo

El sistema solicita al usuario el código del arreglo y el nuevo precio que se desea asignar. Si el código existe en el diccionario **bodega**, el sistema actualiza su precio. Si el código no existe, informa al usuario. Al terminar, pregunta: *"¿Desea actualizar otro precio (s/n)?"*: si la respuesta es "s", el proceso se repite; si es "n", el programa vuelve al menú principal.

Para implementar esta opción:

Define una función **buscar_codigo(codigo)** que recorra el diccionario y retorne True si el código existe, o False si no existe.

Define una función **actualizar_precio(codigo, nuevo_precio)** que debe:

- Verificar la existencia del código (se recomienda invocar a **buscar_codigo** internamente para evitar duplicar la lógica de búsqueda).
- Si el código existe, actualizar el precio en el diccionario y retornar True.
- Si el código no existe, retornar False.

El **programa principal** debe llamar a **actualizar_precio** y, basándose exclusivamente en el valor booleano retornado, decidir qué imprimir: "Precio actualizado" si fue True, o "El código no existe" si fue False.

MALETA
DIDÁCTICA

DuocUC[®]

Recordar que el nuevo_precio debe ser un valor entero positivo y la validación del código no debe distinguir mayúsculas y minúsculas.

Opción 4 — Agregar arreglo

El sistema solicita al usuario todos los datos del nuevo arreglo: código, nombre, tipo, color principal, tamaño, si incluye tarjeta, temporada, precio y unidades. Antes de crear el registro, **cada dato es validado de forma independiente**. Si algún dato no cumple su condición, el sistema informa al usuario y **no registra el arreglo**. Solo cuando todos los datos son válidos y el código no existe previamente, el sistema agrega el registro en ambos diccionarios.

La siguiente tabla resume las condiciones que debe cumplir cada campo:

Campo solicitado	Condición de validación
código	No vacío ni solo espacios en blanco, y que no exista ya en los diccionarios
nombre	No vacío ni solo espacios en blanco
tipo	No vacío ni solo espacios en blanco
color_principal	No vacío ni solo espacios en blanco
tamaño	Debe ser exactamente 'S', 'M' o 'L'
incluye_tarjeta	El usuario ingresa 's' o 'n'. El sistema almacena True si es 's', False si es 'n'
temporada	No vacío ni solo espacios en blanco
precio	Número entero mayor que cero
unidades	Número entero mayor o igual a cero

Para implementar esta opción:

Define una función de validación independiente para cada campo de la tabla anterior. Cada función recibe únicamente el dato a validar, aplica su condición y retorna **True** si es válido o **False** si no lo es. Los mensajes de error **no** se muestran dentro de las funciones de validación.

En el **programa principal**, al elegir esta opción, se solicitan los datos al usuario y se llama a cada función de validación. Si alguna retorna **False**, el programa muestra el mensaje de error correspondiente y no registra el arreglo.

Solo si todas las validaciones retornan **True**, el programa llama a la función **agregar_arreglo(codigo, nombre, tipo, color_principal, tamaño, incluye_tarjeta, temporada, precio, unidades)**, que agrega el registro en ambos diccionarios y retorna **True**. Si el código ya existía, retorna **False**. El programa principal muestra: "Arreglo agregado" o "El código ya existe" según corresponda.

Opción 5 — Eliminar arreglo

El sistema solicita el código del arreglo que se desea eliminar. Si el código existe, elimina el registro en **ambos diccionarios** (arreglos y bodega) e informa que la operación fue exitosa. Si el código no existe, informa al usuario.

Para implementar esta opción:

Reutiliza la función **buscar_codigo(codigo)** definida anteriormente para verificar la existencia del arreglo.

Define una función **eliminar_arreglo(codigo)** que debe:

- Verificar la existencia del código (se recomienda invocar a buscar_codigo internamente para evitar duplicar la lógica de búsqueda).
- Si el código existe, eliminar el registro en ambos diccionarios y retornar True.
- Si el código no existe, retornar False.

El **programa principal** debe llamar a eliminar_arreglo y, basándose exclusivamente en el valor booleano retornado, decidir qué imprimir: "Arreglo eliminado" si fue True, o "El código no existe" si fue False.

Recordar que la validación del código no debe distinguir mayúsculas y minúsculas.

Opción 6 — Salir

El sistema termina la ejecución de forma limpia. El ciclo del menú se detiene y el programa finaliza mostrando el mensaje: *"Programa finalizado."*

Para implementar esta opción:

Esta opción no requiere función adicional. El programa principal es responsable de detener el ciclo del menú y mostrar el mensaje de cierre.

3. Ejemplo de ejecución

A continuación, se muestra un ejemplo representativo de cómo debe funcionar el programa. Los datos en **negrita** son valores ingresados por el usuario:

```
===== MENÚ PRINCIPAL =====
1. Unidades por tipo de arreglo
2. Búsqueda de arreglos por rango de precio
3. Actualizar precio de arreglo
4. Agregar arreglo
5. Eliminar arreglo
6. Salir
=====

Ingrese opción: 1
Ingrese tipo de arreglo a consultar: RAMO
El total de unidades disponibles es: 20

Ingrese opción: 2
```

Ingrese precio mínimo: hola

Debe ingresar valores enteros

Ingrese precio mínimo: 10000

Ingrese precio máximo: 25000

Los arreglos encontrados son: ['Caja Noche--FLO6', 'Centro Mesa--FLO4',
'Ramo Primavera--FLO1']

Ingrese opción: 3

Ingrese código del arreglo: S010

Ingrese nuevo precio: 19990

El código no existe

¿Desea actualizar otro precio (s/n)? : n

Ingrese opción: 4

Ingrese código del arreglo: FLO10

Ingrese nombre: Ramo Aurora

Ingrese tipo: ramo

Ingrese color principal: celeste

Ingrese tamaño (S/M/L): S

¿Incluye tarjeta? (s/n): s

Ingrese temporada: todo_año

Ingrese precio: 14990

Ingrese unidades: 7

Arreglo agregado

Ingrese opción: 6

Programa finalizado.